Муниципальное автономное общеобразовательное учреждение
города Хабаровска «Лицей инновационных технологий»

Тема «Задача о картинной галерее»
Номинация «Математическая грамотность»
Направление «Информационные технологии в современной жизни»

Исследовательская (проектная) работа

Выполнил:

Емельяненко Владимир Фёдорович,
ученик 8Б класса

Руководители проекта:

Дубинская Ирина Александровна,
учитель математики;

Гончаренко Наталья Николаевна,
учитель информатики и ИКТ

г. Хабаровск
2026 год

Оглавление

Глава 1. Введение ... 3
Раздел 1.1. Предисловие ... 3
Раздел 1.2. Актуальность темы ... 3
Раздел 1.3. Проблема. Абстрактная формулировка ... 3
Раздел 1.4. Цель ... 3
Раздел 1.5. Задачи ... 3
Раздел 1.6. Продукт ... 4
Пункт 1.6.1. Описание ... 4
Пункт 1.6.2. Актуальность продукта ... 4
Пункт 1.6.3. Продвижение, рекомендации по использованию и целевая аудитория 4
Глава 2. (Теоретическая часть) Теории, теоремы и решения ... 5
Раздел 2.1. Проблема. Формальная формулировка ... 5
Раздел 2.2. Некоторые очевидные выводы ... 5
Раздел 2.3. Решение Стива Фиска ... 5
Раздел 2.4. Триангуляция многоугольника (без добавления новых вершин) ... 6
Пункт 2.4.1. Некоторые определения [4] ... 7
Пункт 2.4.2. Метод отрезание ушей [5] ... 7
Раздел 2.5. Трираскраска вершин многоугольника [6] ... 9
Раздел 2.6. Выбор группы с наименьшим числом вершин [8] ... 10
Глава 3. Практическая часть ... 11
Раздел 3.1. Триангуляция и трираскраска в коде ... 11
Пункт 3.1.1. Триангуляция [8] ... 11
Пункт 3.1.2. Трираскраска ... 11
Раздел 3.2. Продвижение продукта ... 13
Глава 4. Заключение и напутствие ... 14
Список литературы ... 15
Глава 5. Приложение ... 16
Раздел 5.1. Похожие задачи и вариации данной ... 16
Раздел 5.2. Некоторые решения ... 16
Раздел 5.3. Различные алгоритмы триангуляции ... 16

Глава 1. Введение

Раздел 1.1. Предисловие

Сегодня камеры видеонаблюдения играют решающую роль в поддержании безопасности, мониторинге окружающей среды и инспекции инфраструктуры. К примеру, в микрорайоне «Пятая площадка» г. Хабаровска после установки камер видеонаблюдения преступность снизилась на 80% [1].

Однако каждая камера — дорогостоящее оборудование, требующее затрат на содержание и обслуживание. В этом реферате будет рассмотрена проблема минимизации числа камер видеонаблюдения в здании без потери безопасности.

Раздел 1.2. Актуальность темы

Оптимальная расстановка более важна, чем может показаться на первый взгляд. По данным исследования [2] при оптимальной расстановке камер видеонаблюдения затраты снижаются на $30 - 70\%$ по сравнению с нативным расположением.

Раздел 1.3. Проблема. Абстрактная формулировка[1]

Была сформулирована в 1973 Виктором Кли.

Представив себя владельцем картиной галереи, вы столкнётесь с необходимостью установки камер[2]. Вы опасливый человек, поэтому вам нужно быть *полностью уверенным*, что *любое место в галереи* просматривается *хотя бы одной камерой.* К вашему счастью, камеры могут просматривать всё пространство вокруг себя на $360°$ в *каждый момент времени*. К вашему несчастью, системы можно монтировать только в углах галереи. Как и любому другому расчетливому человеку, вам нужно сократить затраты до минимума.

При факте, что в галерее нет отдельно стоящих стен, какова *верхняя граница* для числа охранников в общем случае?

Раздел 1.4. Цель

Реализовать продукт как обучающий материал в кругу целевой аудитории, *продвигать* и *совершенствовать* его на основе полученной обратной связи.

Раздел 1.5. Задачи

1. *Найти* некоторые способы доказательства в сети из различных источников.

---

[1] Формальная формулировка в геометрических терминах будет приведена позже (см. Проблема. Формальная формулировка).

[2] В оригинальной версии задачи говорится о смотрителях. В современной действительности актуальней будут камеры видеонаблюдения, потому в дальнейшем будем оперировать именно ими.

2. *Выбрать один* способ, использующий сравнительно (с другими способами) меньшее количество вспомогательных действий, более простой для понимания.
3. *Описать*, по возможности *иллюстрируя*, все шаги доказательства, дополняя теоретической базой, которая выходит за рамки школьной программы.
   3.1. *Представить* реферат в качестве ИИП.
4. *Реализовать* анимацию (см. Продукт) как метод наглядного, *упрощённого* доказательства.
   4.1. *Найти* в сети некоторые варианты алгоритмов в `псевдокоде`
   4.2. *Реализовать* алгоритмы в исполняемом `Python` коде и применить их в анимации
5. *Продвигать* продукт в круге целевой аудитории.
   5.1. *Представить* анимацию в качестве продукта на ИИП.
   5.2. *Выложить* алгоритмические реализации, код анимации и комментарии к нему на `GitHub`.

Раздел 1.6. Продукт

Пункт 1.6.1. *Описание*

Итоговым продуктом является анимация, которая создана на `Python` с использованием библиотеки `manim`[1]. Она является, в большей степени, не строгим доказательством теоремы (которое описано в данном реферате), но *наглядным интуитивным объяснением*. Я не старался приводить множество формул и доказательств, а продемонстрировать суть методов, использованных при доказательстве (см. Доказательство Стива Фиска)

Пункт 1.6.2. *Актуальность продукта*

По данным исследования [3], наглядное анимирование (см. Продукт) способно увеличить понимание абстрактных терминов на $25 - 40\%$. Это свидетельствует о важности закрепления теоретической части реферата анимацией.

Пункт 1.6.3. *Продвижение, рекомендации по использованию и целевая аудитория*

Код анимации, и она сама, объяснение работы некоторых функций, использованных при создании анимации, загружено на `GitHub`. Другие разработчики могут использовать мой код как основу собственных способов триангуляции, расчета зоны видимости и создания алгоритмов трёхцветной раскраски.

[1] `Python manim (mathematical animations)` — популярная библиотека для математических анимаций от создателя YouTube-канала 3Blue1Brown.

Глава 2. (Теоретическая часть) Теории, теоремы и решения

Раздел 2.1. Проблема. Формальная формулировка

Многоугольник P, представление в виде графа которого имеет одну компоненту связности, определяется набором n вершин $v_1,\ v_2,\ v_3,\ \dots,\ v_n$ и n рёбер $v_1v_2,\ v_2v_3,\ \dots,\ v_{n-1}v_n,\ v_nv_1$, так, что ни одна непоследовательная пара рёбер не имеет общей точки. Такой многоугольник назовём *простым*.

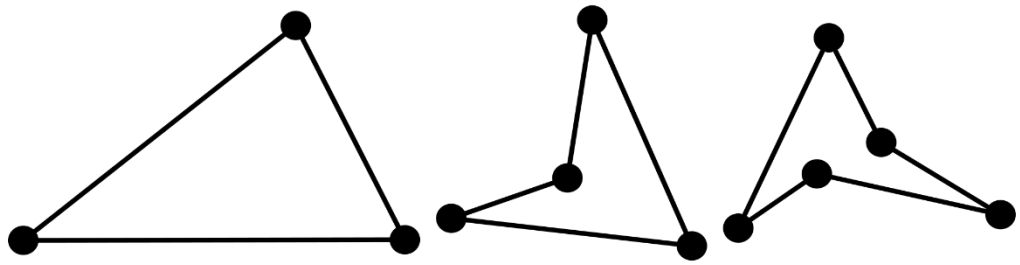
Вершина $x \in P$ *видит* точку $y \in P$, если отрезок $xy \in P$.

Обозначим $g(n)$ как достаточное количество вершин, которое нужно задействовать, чтобы видеть изнутри P любую $y \in P$.

В общем случае, чему равно $\mathrm{g}(n) \to min$?

Раздел 2.2. Некоторые очевидные выводы

Проведя некоторое количество опытов с самыми простыми фигурами, можно с уверенностью говорить, что $g(3) = 1$. Так же $\mathrm{g}(4) = 1$, однако $\mathrm{g}(5) = 1$ уже не так очевидно и придётся проделать больше экспериментов.



$n = 3$ $n = 4$ $n = 5$

Рис. 1. Примеры простейших галерей

Можно заметить, что для любой выпуклой фигуры всегда достаточно задействовать лишь 1 вершину

Теорема. Для выпуклого P всегда необходимо и достаточно задействовать 1 вершину.

Доказательство. Предположим, что требуется больше 1 вершины $\Rightarrow \exists x, y: xy \notin P \Rightarrow P$ *— невыпуклый — противоречие.* ■

Раздел 2.3. Решение Стива Фиска

Решение Стива Фиска, оказавшееся одним из самых коротких и простых в понимании, было предложено незадолго после авторского решения — решения

Ва́цвала Хва́тала. В этом реферате будет подробно описан способ решения Фиска. Начиная, приведём алгоритм:

1. *Триангулируем* P без добавления новых вершин.
2. *Окрасим* вершины P в три цвета так, чтобы вершины каждого треугольника триангуляции были окрашены разными цветами.
3. Множество всех вершин цвета h просматривает весь P по двум причинам: во-первых, вершина цвета h есть во всех треугольниках; во-вторых, треугольник — выпуклая фигура, а по доказанной теореме для такого случая достаточно задействовать 1 вершину.
4. Количество вершин каждого цвета $\approx n/3$. Существует цвет с количеством вершин $\lfloor n/3 \rfloor$, поэтому $\lfloor n/3 \rfloor$ *всегда достаточно.*

Конец алгоритма

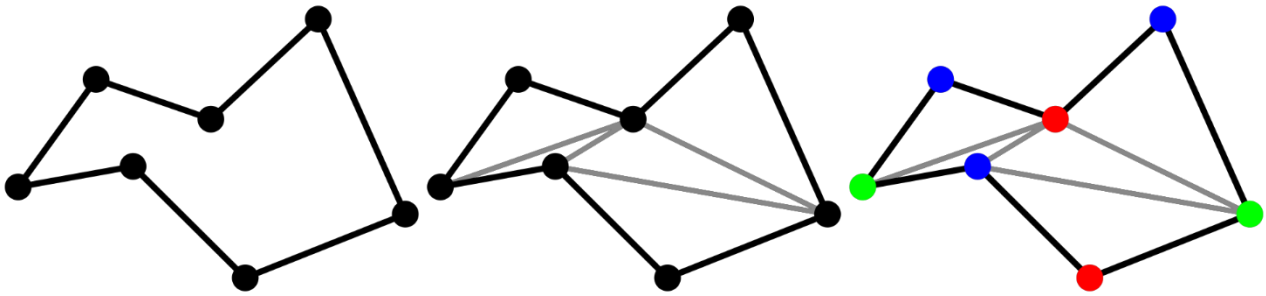


Рис. 2. Алгоритм Стива Фиска. Слева направо: 1) изначальный многоугольник; 2) триангулированный многоугольник; 3) трираскрашенный многоугольник

В доказательстве Фиска три ключевых действия: 1) триангуляция; 2) трираскраска вершин треугольников; 3) выбор цвета с наименьшим числом вершин. Оставшаяся часть теоретической главы будет посвящена именно этим трём действиям, по разделу на каждое.

Раздел 2.4. Триангуляция многоугольника (без добавления новых вершин)

Сама задача *триангуляции многоугольника* рассматривается в многих направления вычислительной геометрии, особенно в компьютерной графике, где работа с геометрическими примитивами серьёзно облегчает вычисления.

Существует множество различных способов триангуляции, но мы разберём самый простой и наглядный — *метод отрезания ушей.*

Пункт 2.4.1. Некоторые определения [4]

Определение 1. Триангуляция – планарный[1] граф, все внутренние области которого — треугольники.

Посмотрите на рисунок справа. Очевидно, у левой триангуляции диагонали избыточно длинные. Итак, правая триангуляция является *оптимальной*, чего нельзя сказать о соседней.

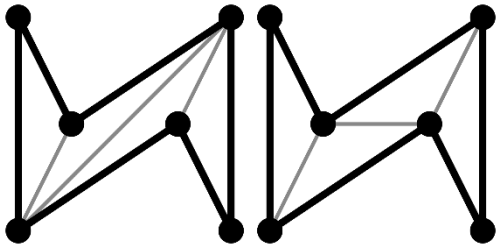


Рис. 3. Неоптимальная (слева) и оптимальная (справа) триангуляция

Определение 2. Триангуляция называется *оптимальной*, если сумма длин всех новых построенных рёбер минимальна среди всех возможных триангуляций, построенных на тех же исходных точках.

Пункт 2.4.2. Метод отрезание ушей [5]

Метод отрезания ушей, как следует из названия, заключается в постепенном «отрезании» треугольников от многоугольника, который в процессе «сжимается», теряя вершины, и в конце концов сам становится треугольником.

Дальше описано более формальное описание этого процесса.

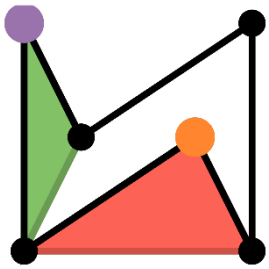


Рис. 4. Главная вершина (фиолетовая), являющаяся ухом, а также главная вершина (оранжевая), не являющаяся ухом.

Некоторые определения

Определение 1. Вершина v_i называется *главной вершиной*, если диагональ $v_{i-1}v_{i+1}$ пересекает P только при вершинах v_{i-1}, v_{i+1} соответственно.

Определение 2. Главная вершина v_i простого многоугольника P называется *ухом*, если диагональ $v_{i-1}v_{i+1}$ полностью лежит внутри P.

Определение 3. Два уха v_i и v_j называются непересекающимися, если они не имеют общих точек.

Некоторые теоремы

Триангуляционная теорема. Каждый простой многоугольник из n вершин может быть разделён на $n-2$ треугольников путём добавления $n-3$ рёбер.

Доказательство. [6] Теорема тривиально верна для $n = 3$, так что пусть $n \geq 4$

[1] Планарный граф – граф, который можно изобразить на плоскости без пересечения его внутренних рёбер [10]

Выберем точку v_2 такую, что $\angle v_1 v_2 v_3 < 180°$.

Случай 1. Если $v_1 v_3$ полностью лежит внутри P, то пусть $\mathrm{d} = v_1 v_3$.

Случай 2 (см. рис. 5). Если $v_1 v_3$ не полностью лежит внутри P. Тогда существует как минимум одна вершина P, лежащая внутри $\triangle\, v_1 v_2 v_3$. Из всех точек, лежащих внутри $\triangle\, v_1 v_2 v_3$, выберем лежащую ближе всего к v_2 точку x. Проведём прямую AH, такую что $\mathrm{AH} \parallel v_1 v_3$ и $x \in AH$. Внутри $\triangle\, A v_2 \mathrm{H}$ гарантированно не лежит ни одной точки P (иначе мы выбрали x неправильно!). Пусть $\mathrm{d} = v_2 \mathrm{x}$.

В обоих случаях d делит P на два меньших многоугольника P_1 и P_2. Если у P_i есть n_i вершин, $\mathrm{i} = \{1,\ 2\}$, тогда $n_1 + n_2 = \mathrm{n} + 2$, так как мы посчитали начало и конец d (точки x и v_2) дважды. Очевидно, $n_i \geq 3$, $\mathrm{i} = \{1,\ 2\}$, и это в совокупности означает что $n_i < \mathrm{n}$, $\mathrm{i} = \{1,\ 2\}$.

Нахождение d для каждого нового P_i приводит к триангуляции P из

$(n_1 - 2) + (n_2 - 2) = (n_1 + n_2) - 4 = n + 2 - 4 = n - 2$ <u>*треугольников*</u> и
$(n_1 - 3) + (n_2 - 3) + 1 = (n_1 + n_2) - 6 + 1 = \mathrm{n} + 2 - 6 + 1 = \mathrm{n} - 3$ <u>*диагоналей*</u>,
включая d. ■

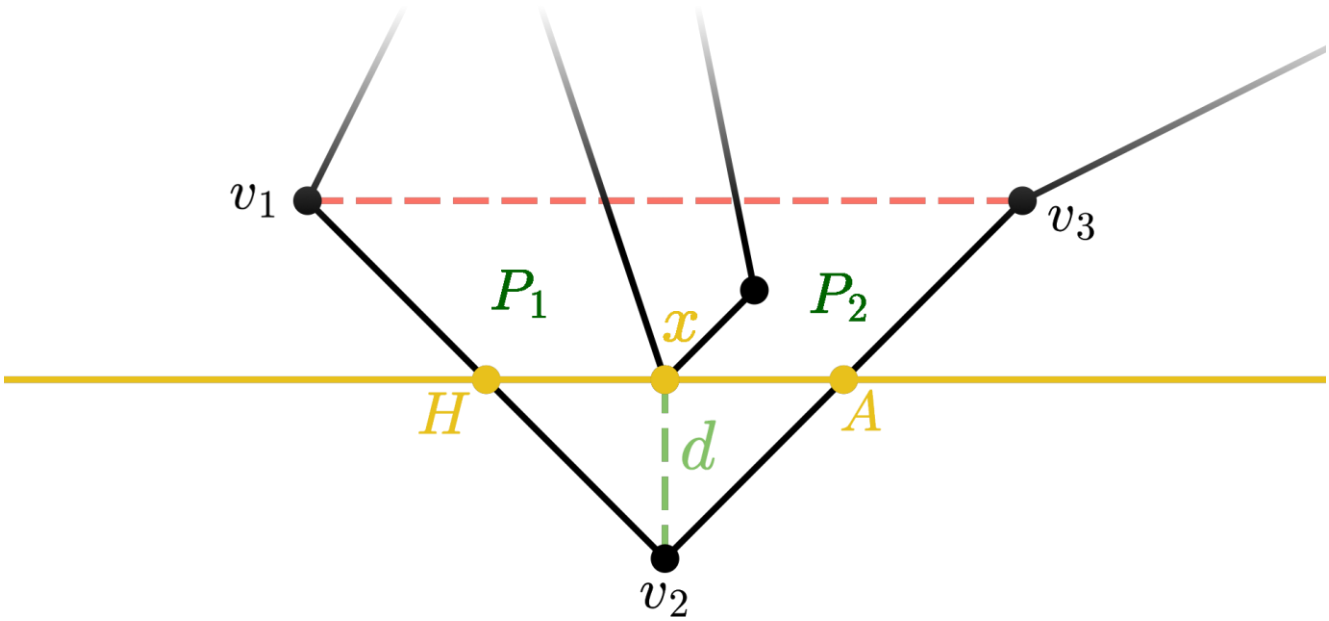


Рис. 5. Случай 2 триангуляционной теоремы

Следствие 1. Сумма внутренних углов многоугольника равна $(n - 2) * 180°$

Доказательство. [6] Каждый из $\mathrm{n} - 2$ треугольников добавляет $180°$ к общей сумме углов. ■

Следствие 2. В любом многоугольнике есть как минимум три угла градусной меры $< 180°$ $\Leftrightarrow$ как максимум $n - 3$ угла градусной меры $> 180°$

Доказательство. Пусть существует $\geq n - 2$ углов градусной меры $> 180°$. Сумма этих углов $> (n - 2) * 180°$. Однако сумма внутренних углов многоугольника равна $(n - 2) * 180°$ – *противоречие*. ■

Теорема о двух ушах. Каждый простой многоугольник, не считая треугольника, имеет как минимум два непересекающихся уха.

Доказательство. [7] Выберем точку v_2 такую, что $\angle v_1 v_2 v_3 < 180°$ (по Следствию 2 такая точка всегда найдётся)

Случай 1. Если $v_1 v_3$ полностью лежит внутри P, то v_2 является ухом по определению. Отрезав его, мы получим, что P' является либо треугольником, и, следовательно, вторым ухом, либо многоугольником с $n - 1$ вершинами, из которых как минимум два образуют угол $< 180°$ (в соответствии со Следствием 2). В итоге из трёх ушей как минимум два непересекающиеся.

Случай 2 (см. рис. 5 (выше)). Если $v_1 v_3$ не полностью лежит внутри P. Тогда существует диагональ d (ссылаясь на Триангуляционную теорему), делящая изначальный P на два меньших многоугольника P_1 и P_2. В каждом из ник есть как минимум по одному уху, в сумме получаем 2 непересекающихся. ■

Из всех вышеперечисленных теорем отрезание уха для каждого нового P' приведёт к триангуляции P.

<u>Вывод о методе триангуляции путём отрезания ушей</u>

Данный метод является самым интуитивно понятным способом триангуляции. Конечно, у него есть и свои недостатки. Ушной способ триангуляции не всегда возвращает оптимальную триангуляцию, так как очень зависит от порядка обхода точек. Он также не является лучшим по асимптотической сложности. Вследствие этого в приложении к реферату перечислены ссылки на некоторые другие труды по способам триангуляции (см. Различные алгоритмы триангуляции)

Раздел 2.5. Трираскраска вершин многоугольника [6]

Задав себе вопрос: «Можно ли все вершины триангулированного многоугольника раскрасить в три цвета так, чтобы вершины любого из треугольников были окрашены в три разных цвета?» и проведя некоторые эксперименты, можно понять, что это всегда получается, ведь, раскрасив какой-нибудь треугольник в три цвета,

для всех треугольников, имеющими с данным общую сторону, раскраска будет определена.

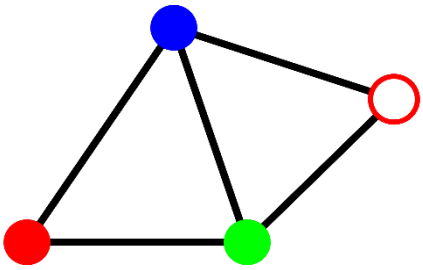


Рис. 6. Слева: трираскрашенный треугольник; справа: треугольник, имеющий с раскрашенным общую сторону и для которого раскраска очевидна.

Теорема о 3-х цветной раскраске. Все вершины триангуляции простого многоугольника можно раскрасить в три цвета так чтобы у любого треугольника все вершины были разных цветов.

Доказательство (см. рис. 6). Будем отрезать у многоугольника уши, пока не останется один единственный треугольник. Раскрасим его вершины в три цвета. Теперь обратим процесс триангуляции вспять, то есть вернём предыдущее ухо. Его можно однозначно раскрасить, так как оно имеет две общие (уже раскрашенные) вершины с предыдущим ухом (ссылаясь на теорему о двух ушах). Повторение этого процесса приведёт к требуемой трёхцветной раскраске.■

Раздел 2.6. Выбор группы с наименьшим числом вершин [8]

Теперь есть три группы разных цветов, состоящие из $\sim n/3$ вершин. Пусть T_1, T_2, T_3 являются таковыми группами. Тогда существует неравенство, без потери общности скажем, что оно имеет вид $T_1 \leq T_2 \leq T_3$. Выберем $T_i \leq n/3$. Это приводит к решению $\lfloor n/3 \rfloor$.

Глава 3. Практическая часть

Раздел 3.1. Триангуляция и трираскраска в коде

Пункт 3.1.1. Триангуляция [8]

Псевдокод

```
1 def triangulate(polygon):
2     # Объявляем переменную триангулированного многоугольника
3     triangulated_polygram = Polygram()
4
5     # Пока многоугольник не стал треугольником
6     while polygon is not Triangle:
7         # ... проверяем все вершины
8         for vertex in polygon.get_vertices():
9             possible_ear = Triangle(prev_vertex, vertex, next_vertex)
10            # проверяем, является ли ухом
11            if vertex is Convex and polygon.get_vertices() is not inside possible_ear:
12                # если да, отрезаем и добавляем в триангуляцию
13                polygon.degenerate(possible_ear)
14                triangulated_polygram.add(possible_ear)
15
16    return triangulated_polygram
```

Оценка временной/пространственной сложности

Программа итерируется по вершинам многоугольника (цикл в 8-й строке), что требует $O(n)$ времени. Далее 2-я проверка в 11-й строке на отсутствие вершин многоугольника внутри уха также займёт $O(n)$. Затем функция `degenerate`, представляющая из себя простое удаление вершины из массива вершин (что при традиционном подходе займёт лишнее $O(n)$) может быть реализована с помощью односвязного списка и быть константной по времени.

Суммарно, программа будет исполнена за $O(n^2)$ времени. На хранение списка треугольников потребуется $O(n)$ памяти.

Пункт 3.1.2. Трираскраска

Псевдокод

```
1 def tricolor(triangles, polygon_verts_count):
2     def third_non_colored(triangle):
3         """
4         Некоторая функция, которая возвращает
5         третью не закрашенную вершину
6         треугольника И целевой цвет этой вершины.
7
8         Если же в треугольнике количество
9         уже закрашенных вершин не равно 2,
10        возвращает None
11        """
12
13    # Объявляем группы цветов
14    list color_a
15    list color_b
16    list color_c
17
18    # Окрашиваем все вершины первого треугольника,
19    # чтобы группы не были пустыми
20    add v1 of triangle1 into color_a
21    add v2 of triangle1 into color_b
22    add v3 of triangle1 into color_c
23
24    # Окрашиваем остальные вершины
25    len_sum = len(color_a) + len(color_b) + len(color_c)
26    while len_sum < polygon_verts_count:
27        for triangle in triangles:
28            non_colored = third_non_colored(triangle)
29            if non_colored is not None:
30                get target_color, vertex from non_colored
31                add vertex of triangle in target_color
32
33    return color_a, color_b, color_c
```

Оценка временной/пространственной сложности

В основе своей программа итерируется по всем треугольникам, ища треугольники с 2 закрашенными вершинам. В худшем случае мы лишь несколько раз перезапускаем цикл в 27-й строке, поэтому временная сложность равна $O(n)$.

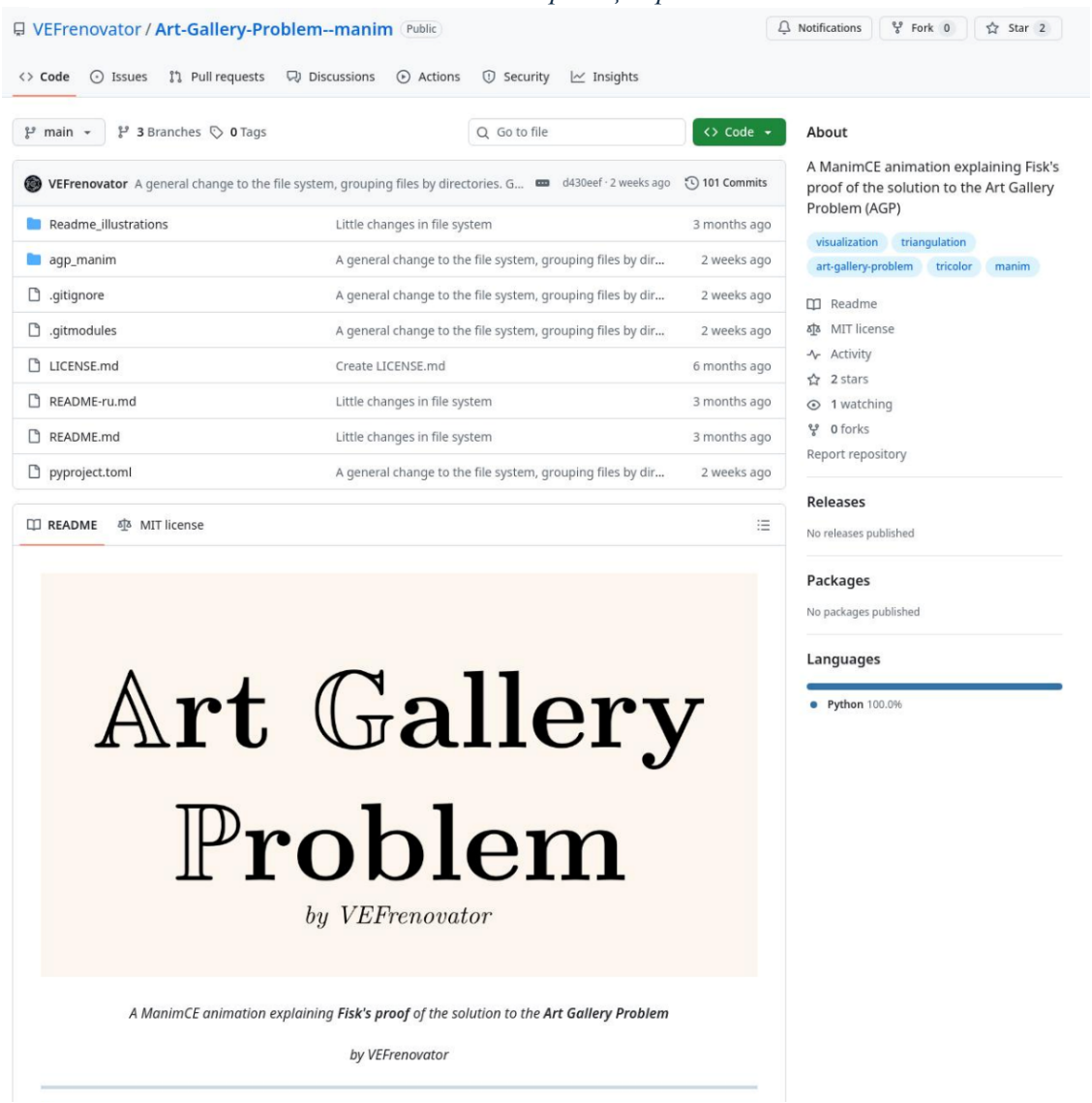
Программа запоминает цвета каждой вершины, на что уходит $O(n)$ памяти.

Раздел 3.2. Продвижение продукта

Вся поддержка проекта, улучшение анимации и другие изменения вносятся в репозиторий Art-Gallery-Problem--manim. В действительности анимация не сводится лишь к написанию кода для вывода действий на экран, но требует создание множества вспомогательных модулей, таких как система обработки оглавления, разбиение на презентативные сцены, отображение знаков авторских прав и др. В сумме проект насчитывает ~2000 строк python-кода. Такая разносторонность может привлечь внимание других разработчиков со всего мира (описание составлено на английском). В подтверждение этому упомянем, что за время разработки проекта страницу репозитория посетило чуть больше 1000 пользователей, 200 из которых скопировали его к себе для локальной работы над кодом. Отдельно хотелось бы поблагодарить разработчика с ником emhayki, который позволил позаимствовать его наработки анимирования.

Всё это говорит о том, что продукт по-настоящему полезен аудитории, а его дальнейшее развитие, написание документации и исправление багов *необходимо*.

Рис. 7. Заглавная страница проекта



Глава 4. Заключение и напутствие

Заключая, можно говорить о достижении всех поставленных теоретических и практических задач. Удалось узнать некоторые из способов решения и описать одно, которое, к слову сказать, занесено в книгу «Доказательства из книги» как одно из самых наглядных и показательных. В ходе доказательства был изучен материал, выходящий за рамки школьной программы, например триангуляция простого многоугольника, некоторые элементы теории графов и анализ асимптотической сложности алгоритма. Удалось реализовать анимацию и выдвинуть её в свет, донеся продукт до целевой аудитории. Продвижение продукта принесло свои плоды, была получена обратная связь от аудитории.

Если Вы заинтересовались темой и хотите узнать больше, то Вам могут быть интересны похожие геометрические задачи и вариации данной, список которых приведён в приложении. По любым предложениям и вопросам пишите на странице Discussions на GitHub'е проекта.

Я искренне благодарен Вам за прочтение реферата.

Список литературы

1. EAOMedia. Сергей Кравчук: В систему "Безопасный город" внесём все хабаровские точки видеонаблюдения Ноябрь 2023.
2. Kumar Y., Bollapragada R., Leibowicz B. Efficient mathematical programming formulation and algorithmic framework for optimal camera placement // Arxiv. 2024. Режим доступа: https://arxiv.org/pdf/2411.17942
3. Sandra Berney, Mireille Bétrancourt. Does animation enhance learning? A meta-analysis // ScienceDirect. Режим доступа: https://www.sciencedirect.com/science/article/abs/pii/S0360131516301336?via%3Dihub
4. Скворцов А.В. М.Н.С. Алгоритмы построения и анализа триангуляции. Томск: Издательство Томского Университета, 2006.
5. Ian Garton. Ear Cutting for Simple Polygons // McGill. Режим доступа: https://cgm.cs.mcgill.ca/~godfried/teaching/cg-projects/97/Ian/introduction.html
6. O'Rourke J. Art Gallery Theorems and Algorithms. Нью-Йорк: Oxford University Press, 1987.
7. Meisters G. H. Polygons have ears. Линкольн: University of Nebraska, 1975. 648-651 pp.
8. Fisk S. A Short Proof of Chvatal's Watchman Theorem // В кн.: A Combinatorial Theorem in Plane Geometry / ред. Chvatal V. Брансуик: Department of Mathematics, Bowdoin College, 1975. С. 39-41.
9. Ian Garton. The Two-Ears Theorem // Kent Stage University. Режим доступа: http://personal.kent.edu/~rmuhamma/Compgeometry/MyCG/TwoEar/two-ear.htm
10. Яковлев И. В. Планарные графы. Формула Эйлера. Учебный материал.

Глава 5. Приложение

Раздел 5.1. Похожие задачи и вариации данной

- **Задача о защите крепости.** Смысл в том, чтобы защитить многоугольник камерами извне.
- **Задача о картинной галереи, однако…**
  - …все стены расположены под прямыми углами.
  - …в галереи есть «колонны». Ввиду имеются многоугольники с дырами, то есть те, представление в виде графов которых имеет более одной компоненты связности.
  - …галерея трёхмерна или имеет большее количество измерений.

Раздел 5.2. Некоторые решения

- **Решение Ва́цвала Хва́тала, создателя задачи.** «A Combinatorial Theorem in Plane Geometry»

Раздел 5.3. Различные алгоритмы триангуляции

Конечно, есть и другие способы триангуляции. Ссылки на некоторые из них я оставлю ниже и предложу читателю ознакомиться с ними при желании и возможности.

- **Оптимизированная триангуляция методом отрезания ушей за $O(n^2)$ в среднем случае.**
  - [*англ.*] https://www.geometrictools.com/Documentation/Triangulation-ByEarClipping.pdf // стр. 3 // Dabid Eberly
- **Разбиение на монотонные многоугольники.** Временная сложность $\mathrm{O}(\mathrm{n}\log n)$.
  - [*англ.*] https://www.science.smith.edu/~jorourke/books/ArtGalleryTheorems/Art_Gallery_Full_Book.pdf // стр. 14 – 19 // эл. версия книги «Art Gallery Theorem and Algorithms», Joseph O'Rourke // ISBN 0-19-503965-3
- **Триангуляция Чазелла.** Временная сложность $\mathrm{O}(n)$. Сложен в понимании и использует множество вспомогательных процедур.
  - [*англ.*] https://www.cs.princeton.edu/~chazelle/pubs/polygon-triang.pdf // [эл. версия книги «Triangulation a Simple Polygon in Linear Time», B. Chazell